



Random Number Generation and Monte Carlo

What You Will Learn Today

- **By the end, you can:**
 - Explain $U(0,1)$, pseudo-random numbers, and seeds in plain language.
 - Calculate a small LCG sequence one step at a time.
 - Turn a uniform value into a simulated time or demand.
 - Repeat many trials and interpret a Monte Carlo result.

Why Simulation Needs Random Numbers

A coffee shop does not receive the same number of customers every hour.

Some customers arrive early. Others arrive late.

A simulation needs different possible arrivals, service times, and demands.

Random numbers let the computer create those possible situations.

Guiding question: How can a computer create uncertainty in a repeatable way?

Think of $U(0,1)$ as a Raffle Ticket

Imagine drawing a ticket from a drum.

The ticket contains one decimal from 0 to 1, such as 0.14 or 0.72.

Every equally sized part of the interval has the same chance.

The ticket is only a starting value. We later translate it into minutes, customers, or units of demand.

Key idea: U is the raw ingredient.

Every Random Input Follows the Same Pipeline

- GENERATE a decimal U between 0 and 1.
- TRANSLATE U into a useful value, such as a waiting time.
- USE that value in the simulation.
- REPEAT to create many possible outcomes.
- **Remember: Generate → Translate → Use → Repeat.**

Pseudo-Random Means a Repeatable Shuffle

Think of a computer as an automatic card shuffler.

It follows an algorithm, but the results look unpredictable.

This is useful because simulations need many values quickly.

A recorded starting point lets us reproduce the same shuffle later.

Pseudo does not mean fake or poor. It means algorithm-generated.

Worked Seed Example in NumPy

```
import numpy as np
rng1 = np.random.default_rng(142)
rng2 = np.random.default_rng(142)
print(rng1.random(3))
print(rng2.random(3))
# Both print the same three values
```

Pause and Check: Seed

- Suppose two students use the same program and seed 142.
- Will their first three random values match? YES, if the call order is also the same.
- What if one program draws an extra value? Every later draw shifts.
- One-sentence takeaway: The seed controls where the sequence starts.

An LCG Is a Small Number Machine

- Imagine a machine with four controls:
- a multiplies the previous number.
- c adds a small push.
- m sets where the numbers wrap around.
- Z_0 is the starting number, or seed.
- The machine repeats the same recipe to produce the next state.

LCG Step 1: Generate the Next State

1. Generate the next state

$$Z_i = (aZ_{i-1} + c) \bmod m$$

Where:

Z_{i-1} = previous state or previous integer

Z_i = *newly generated state*

a = multiplier

c = increment

m = modulus

$\bmod$ = remainder operation

For example, suppose:

$$a = 5, \quad c = 3, \quad m = 16, \quad Z_0 = 7$$

Generate the first state:

$$Z_1 = (5(7) + 3) \bmod 16$$

The remainder when 38 is divided by 16 is **6**.

LCG Step 2: Convert the State into U

2. Convert the state into a uniform number

$$U_i = \frac{Z_i}{m}$$

Using $Z_1 = 6$ and $m = 16$

$$U_1 = \frac{6}{16} = 0.375$$

Therefore, the first generated uniform random number is:

$$U_1 = 0.375$$

The process then repeats using Z_1 :

$$Z_2 = (5(6) + 3) \bmod 16 = 33 \bmod 16 = 1$$

$$U_2 = \frac{1}{16} = 0.0625$$

So the sequence begins:

Step	Previous state	New state	Uniform output
1	$Z_0 = 7$	$Z_1 = 6$	$U_1 = 0.375$
2	$Z_1 = 6$	$Z_2 = 1$	$U_2 = 0.0625$
3	$Z_2 = 1$	$Z_3 = 8$	$U_3 = 0.500$

Modulo Means Wrap Around

- Think of a clock with positions 0 to 15.
- The calculation gives 38, but the clock has only 16 positions.
- After wrapping around twice, 38 lands on position 6.
- Mathematically: $38 \bmod 16 = 6$.
- **Modulo simply keeps the remainder after division.**

Period Means Steps Before Repetition

A generator eventually returns to an earlier state.

From that point, the same values repeat in the same order.

The number of states before repetition is the period.

A short period is like shuffling only a tiny deck: patterns return too soon.

For real work, use a modern tested generator with a very long period.

Uniformity and Independence Are Different

Uniformity asks: Are values spread fairly across 0 to 1?

Analogy: Every section of a raffle drum receives roughly equal results.

Independence asks: Does one value reveal the next?

Analogy: A good sequence should not alternate low, high, low, high.

A generator should satisfy both.

A Pair Plot Makes Hidden Patterns Visible

Pair each value with the next value: (U_i, U_{i+1}) .

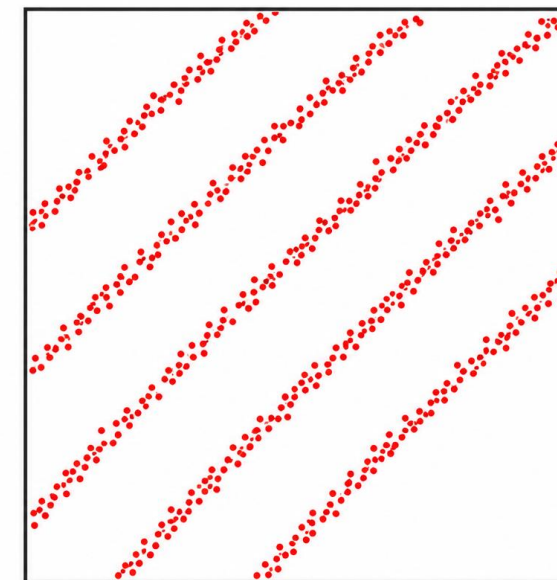
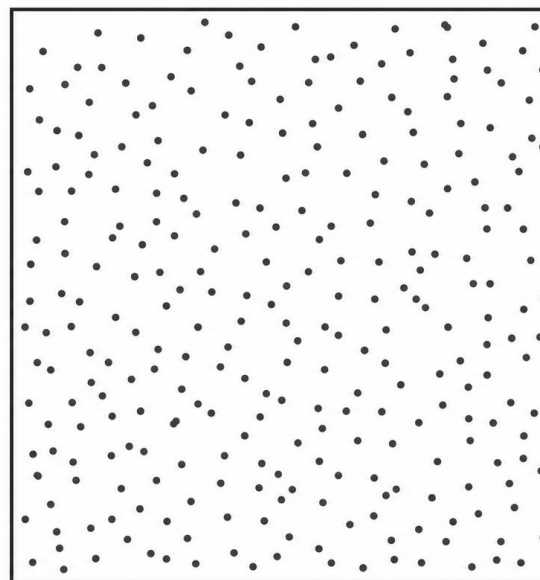
Good sign: points spread naturally across the square.

Warning sign: points form lines, bands, or empty gaps.

A histogram can look fair even when the order contains a pattern.

- Read the visual: left looks random; right is predictable.

LEFT: healthy spread RIGHT: warning bands



NumPy Generator Is the Practical Tool

```
import numpy as np
rng = np.random.default_rng(seed=142)
uniforms = rng.random(5)
counts = rng.poisson(lam=6, size=5)
times = rng.exponential(scale=4, size=5)
print(uniforms)
```

U Is Only the Raw Number

A random variate is one generated observation in real units.

Examples: 2.4 minutes, 7 arrivals, or 55 units of demand.

The computer begins with U between 0 and 1.

A transformation translates U into the distribution required by the model.

Beginner rule: Generate U → transform it → interpret the result.

Inverse Transform Is a Percentile Lookup

- Imagine $U = 0.70$ means the 70th percentile.
- Generate U between 0 and 1.
- Locate that probability position on the distribution.
- Read the matching value X in minutes, customers, or units.
- PowerPoint-safe rule: $X = \text{inverse CDF}(U)$.
- **Meaning: U chooses a position; the inverse CDF translates it.**

Build the Exponential Formula Step by Step

Start: $U = 1 - \exp(-X / \beta)$

1. $1 - U = \exp(-X / \beta)$

2. $\ln(1 - U) = -X / \beta$

3. $X = -\beta \ln(1 - U)$

Terms: beta is the mean time; ln is the natural logarithm.

Result: The formula converts a uniform value into an exponential time.

Worked Example: Generate One Waiting Time

- Given: $U = 0.257$ and mean $\beta = 2$ minutes.
- Rule: $X = -\beta \ln(1 - U)$
- Compute $1 - U$: $1 - 0.257 = 0.743$.
- Take the natural logarithm: $\ln(0.743) \approx -0.297$.
- Multiply by -2 : $X \approx 0.594$ minute.
- **Interpretation: This is one possible waiting time, not the average.**

Cumulative Probabilities Form a Prize Wheel

Each outcome owns a section of the interval from 0 to 1.

Example ranges:

1: 0.000–0.167 2: 0.167–0.500

3: 0.500–0.833 4: 0.833–1.000

If $U = 0.40$, it lands in outcome 2. If $U = 0.80$, it lands in outcome 3.

Rule: Choose the first cumulative probability that reaches U .

Other Methods Follow the Same Big Idea

Box-Muller: two uniform values become normal values.

Analogy: combine two raw ingredients to make a different result.

Acceptance-rejection: propose a candidate, check it, then keep or retry.

Poisson generation: create a count, such as calls per hour.

For this lesson, recognize the purpose. In the lab, use NumPy's tested methods.

Pause and Check: Transforming U

- If $U = 0.40$ and the boundary for outcome 2 is 0.50, which outcome is selected? Outcome 2.
- Does $U = 0.40$ automatically mean 0.40 minute? No.
- What is missing? A transformation into the required unit.
- One-sentence takeaway: U is translated before it enters the model.

Monte Carlo Rehearses Many Possible Futures

One trial represents one possible future.

A simulation repeats the trial hundreds or thousands of times.

The results show what happens often, rarely, or on average.

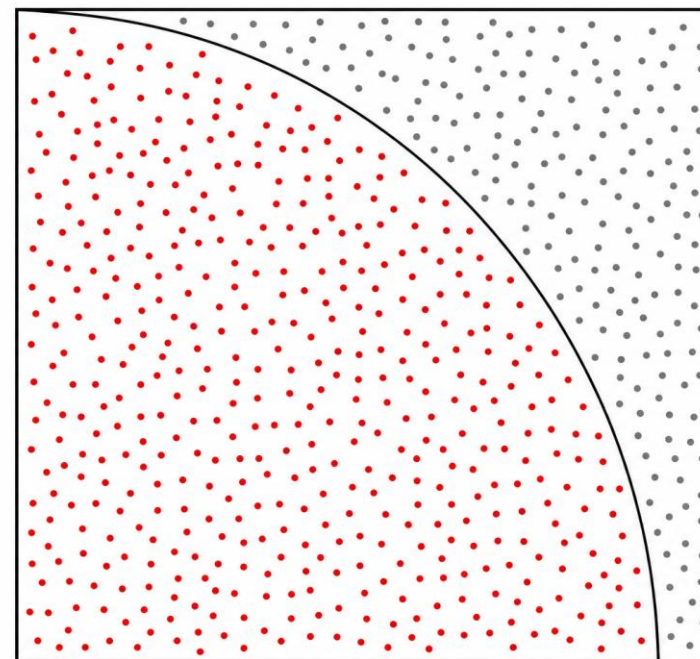
Analogy: rehearse many possible days before making a real decision.

Basic recipe: define → generate → calculate → repeat → summarize → interpret.

Estimate Pi by Throwing Virtual Darts

- Generate a random point (x, y) inside a 1×1 square.
- Check whether $x^2 + y^2 \leq 1$.
- Count the fraction inside the quarter-circle.
- Estimate $\pi = 4 \times \text{inside fraction}$.
- **More virtual darts usually produce a steadier estimate.**

RED: inside quarter-circle GRAY: outside



Code a Reproducible Pi Experiment

```
import numpy as np
rng = np.random.default_rng(142)
n = 100_000
x = rng.random(n)
y = rng.random(n)
inside = x*x + y*y <= 1
pi_hat = 4 * inside.mean()
print(pi_hat) # 3.13096 with seed 142
```

4× the Trials, About Half the Noise

Monte Carlo estimates wiggle because they use random samples.

Typical error decreases like $1 / \sqrt{n}$.

1,000 trials → reference amount of noise

4,000 trials → about half as much noise

16,000 trials → about one quarter as much noise

Important: More trials cannot fix a wrong model.

Lab Story: Can Demand Exceed Capacity?

- A store can serve 50 units per day.
- Possible demand: 35, 40, 45, 50, 55, or 60 units.
- Demand above 50 creates a shortage.
- **Questions:**
 - How often does demand exceed capacity?
 - What is the average shortage?
 - We will answer by simulating many possible days.

Before Coding, Simulate Three Days by Hand

Use the cumulative demand ranges:

35: 0.00–0.10

40: 0.10–0.25

45: 0.25–0.50

50: 0.50–0.75

55: 0.75–0.90

60: 0.90–1.00

Three sample days:

Day	U	Demand	Shortage
1	0.07	35	0
2	0.62	50	0
3	0.94	60	10

Lab Code: Run the Demand Experiment

```
import numpy as np
rng = np.random.default_rng(142)
d = np.array([35, 40, 45, 50, 55, 60])
p = np.array([.10, .15, .25, .25, .15, .10])
sample = rng.choice(d, size=10_000, p=p)
overload = (sample > 50).mean()
shortage = np.maximum(sample - 50, 0).mean()
print(overload, shortage)
```

The Simulation Is Close to the Exact Answer

Using seed 142 and 10,000 trials:

Estimated overload probability = 0.2554

Estimated average shortage = 1.7955 units/day

Exact overload probability = 0.25

Exact average shortage = 1.75 units/day

Interpretation: Small differences are expected sampling error.

Week 4: The Whole Story

- Generate U between 0 and 1.
- Translate U into a meaningful random input.
- Use the input in one simulated trial.
- Repeat many trials.
- Summarize the results and explain the uncertainty.
- **Next week: Use random arrivals and services in discrete-event simulation.**